

Flutter TS Identity Orchestration - User Guide

Overview

The Flutter TS Identity Orchestration plugin enables secure identity verification and authentication flows in your Flutter app using Transmit Security's Identity Orchestration platform.

Requirements

- **Flutter:** 3.0.0 or higher
- **iOS:** 13.0 or higher
- **Android:** API level 21 (Android 5.0) or higher
- **Dart:** 2.17.0 or higher

Native SDK Documentation

For platform-specific implementation details and advanced configuration, refer to the official Transmit Security native SDK documentation:

- **iOS WebAuthn SDK:** [Quick Start Guide](https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_ios)
(https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_ios)
- **Android WebAuthn SDK:** [Quick Start Guide](https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_android)
(https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_android)

These guides provide detailed information about native SDK features, configuration options, and platform-specific implementation requirements.

Installation

1. Add Dependency

Add this to your pubspec.yaml:

```
dependencies:  
  flutter_ts_identity_orchestration:  
    git:  
      url: https://github.com/TransmitSecurity/flutter_ts_identity_orchestration.git  
      ref: 0.0.3
```

Run:

```
flutter pub get
```

2. iOS Setup (If using camera and FaceID)

Add to your ios/Runner/Info.plist:

```
<key>NSCameraUsageDescription</key>
<string>This app needs camera access for identity verification</string>
<key>NSFaceIDUsageDescription</key>
<string>This app uses Face ID for authentication</string>
```

Minimum iOS version in ios/Podfile:

```
platform :ios, '13.0'
```

3. Android Setup (If using camera)

Add to your android/app/src/main/AndroidManifest.xml:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
```

Minimum SDK in android/app/build.gradle:

```
android {
    compileSdkVersion 34
    defaultConfig {
        minSdkVersion 21
        targetSdkVersion 34
    }
}
```

Integration

Maven Configuration

Add Maven repository to both the module and app-level build files:

Module level (android/build.gradle):

```
allprojects {
    repositories {
        google()
        mavenCentral()
        // Add Transmit Security Maven repository
        maven {
            url "https://transmit.jfrog.io/artifactory/transmit-security-gradle-release-local/"
        }
    }
}
```

For Kotlin use:

```
maven {
    url = uri("https://transmit.jfrog.io/artifactory/transmit-security-gradle-release-local/")
}
```

App level (android/app/build.gradle):

```
repositories {  
  google()  
  mavenCentral()  
  // Add Transmit Security Maven repository  
  maven {  
    url "https://repo.transmitsecurity.io/repository/tspublic/"  
  }  
}
```

Android MainActivity Configuration

IMPORTANT: Make sure that your MainActivity inherits from FlutterFragmentActivity instead of FlutterActivity:

```
import io.flutter.embedding.android.FlutterFragmentActivity  
  
class MainActivity: FlutterFragmentActivity() {  
  // Your MainActivity code here  
}
```

This is required for proper plugin functionality and native SDK integration.

Quick Start

1. Import the Package

```
import 'package:flutter_ts_identity_orchestration/flutter_ts_identity_orchestration.dart';
```

2. Initialize the SDK

```
class MyApp extends StatefulWidget {  
  @override  
  _MyAppState createState() => _MyAppState();  
}  
  
class _MyAppState extends State<MyApp> {  
  final _orchestration = FlutterTsIdentityOrchestration();  
  StreamSubscription<Map<String, dynamic>>? _subscription;  
  
  @override  
  void initState() {  
    super.initState();  
    _initializeSDK();  
    _listenToResponses();  
  }  
  
  void _initializeSDK() async {  
    try {  
      // Option 1: Initialize with parameters  
      await _orchestration.initialize(  
        clientId: 'your-client-id',  
        options: {
```

```

    'serverPath': 'https://api.transmitsecurity.io',
    'resource': null,           // Optional: custom resource path
    'pollingTimeout': null,     // Optional: polling timeout in milliseconds
    'locale': 'en',            // Optional: locale setting (default: 'en')
  },
);

// Option 2: Initialize with config file (iOS: plist, Android: auto-detected)
// await _orchestration.initializeSDK(configurationFile: 'TransmitSecurity');

print('SDK initialized successfully');
} catch (e) {
  print('Failed to initialize SDK: $e');
}
}

void _listenToResponses() {
  // CRITICAL: You MUST listen to journeyResponseStream to receive SDK responses
  // This stream provides real-time updates from the identity orchestration journey
  // including user interface steps, authentication results, and error notifications
  _subscription = _orchestration.journeyResponseStream.listen(
    (response) {
      print('Journey response: $response');
      _handleJourneyResponse(response);
    },
    onError: (error) {
      print('Journey error: $error');
    },
  );
}

@override
void dispose() {
  _subscription?.cancel();
  super.dispose();
}
}

```

⚠ CRITICAL: Listen to Journey Responses

Before starting ANY journey, you MUST set up response listening. This is not optional.

The `_listenToResponses()` method is **essential** for your app to function correctly:

- **What it does:** Creates a persistent connection to receive real-time updates from the Transmit Security SDK
- **When to call:** Immediately after SDK initialization, before starting any journeys
- **What it receives:** Authentication results, user interface instructions, error notifications, journey completion status
- **Without it:** Your app will start journeys but never receive responses - users will be stuck

```
// CORRECT: Listen first, then start journey
void initState() {
  super.initState();
  _initializeSDK();
  _listenToResponses(); // ← MANDATORY: Must be called before starting journeys
}

// INCORRECT: Starting journey without listening will result in no responses
void badExample() {
  _orchestration.startJourney(journeyId: 'test'); // Will hang indefinitely!
}
```

3. Start a Journey

```
void _startJourney() async {
  try {
    await _orchestration.startJourney(
      journeyId: 'your-journey-id',
      options: {
        'additionalParams': {'key': 'value'}, // Optional
        'flowId': 'custom-flow-id', // Optional
        'encryptionMode': true, // Optional: enable double encryption
      },
    );
    print('Journey started');
  } catch (e) {
    print('Failed to start journey: $e');
  }
}
```

4. Handle Journey Responses

```

void _handleJourneyResponse(Map<String, dynamic> response) {
  final bool isSuccess = response['success'] == true;

  if (!isSuccess) {
    print('Journey failed: ${response['error']}');
    return;
  }

  final responseData = response['response'];
  final String? stepId = responseData?['journeyStepId'];

  switch (stepId?.toLowerCase()) {
    case 'action:information':
      _showInformationScreen(responseData);
      break;
    case 'login_form':
      _showLoginForm(responseData);
      break;
    case 'action:rejection':
      _showRejectionDialog(responseData);
      break;
    default:
      print('Unhandled step: $stepId');
  }
}

```

5. Submit Responses

```

void _submitResponse(String responseld, Map<String, dynamic>? data) async {
  try {
    final success = await _orchestration.submitClientResponse(
      responseld: responseld,
      data: data,
    );

    if (success) {
      print('Response submitted successfully');
    } else {
      print('Failed to submit response');
    }
  } catch (e) {
    print('Error submitting response: $e');
  }
}

// Example: Submit login credentials
void _submitLogin(String username, String password) {
  _submitResponse('clientInput', {
    'username': username,
    'password': password,
  });
}

// Example: Cancel current step
void _cancelStep() {
  _submitResponse('cancel', null);
}

```

API Reference

1. initializeSDK

Initialize SDK with configuration file.

```
Future<void> initializeSDK({String? configurationFile})
```

Example:

```
await _orchestration.initializeSDK(configurationFile: 'TransmitSecurity.plist');
```

2. initialize

Initialize SDK with parameters.

```
Future<void> initialize({
  required String clientId,
  required Map<String, dynamic> options,
})
```

Parameters:

- `clientId` (String): Your Transmit Security client identifier
- `options` (Map<String, dynamic>): Configuration options
 - `serverPath` (String): Server URL (default: 'https://api.transmitsecurity.io')
 - `resource` (String?): Optional custom resource path
 - `pollingTimeout` (int?): Optional polling timeout in milliseconds
 - `locale` (String?): Optional locale setting (default: 'en')

Example:

```
await _orchestration.initialize(
  clientId: 'your-client-id',
  options: {
    'serverPath': 'https://api.transmitsecurity.io',
    'resource': null,
    'pollingTimeout': 30000, // 30 seconds
    'locale': 'en',
  },
);
```

3. startJourney

Start an identity verification journey.

```
Future<void> startJourney({
  required String journeyId,
  Map<String, dynamic>? options,
})
```

Example:

```
await _orchestration.startJourney(
  journeyId: 'registration-flow',
  options: {
    'additionalParams': {'userType': 'premium'},
    'encryptionMode': true,
  },
);
```

4. submitClientResponse

Submit response during journey flow.

```
Future<bool> submitClientResponse({
  required String responseId,
  Map<String, dynamic>? data,
})
```

Common Response IDs:

- 'clientInput' - Submit user input
- 'cancel' - Cancel current step
- 'fail' - Mark step as failed
- 'resend' - Request resend (e.g., OTP)

Example:

```
// Submit form data
await _orchestration.submitClientResponse(
  responseld: 'clientInput',
  data: {'email': 'user@example.com'},
);

// Cancel flow
await _orchestration.submitClientResponse(
  responseld: 'cancel',
  data: null,
);
```

⚠ Instructions Callback:

When calling `submitClientResponse`, `startMobileApproveJourney`, or `generateDebugPin`, the native SDK may trigger additional instructions through the event stream. These will be delivered via the `journeyResponseStream` with type 'instructions'.

Handling Instructions:

```
_orchestration.journeyResponseStream.listen((response) {
  if (response['type'] == 'instructions') {
    final instructions = response['instructions'];
    print('Instructions received: $instructions');

    // Handle instructions data
    // Instructions format depends on the specific journey step
    // and may contain additional guidance or configuration
  }
});
```

5. generateDebugPin

Generate debug PIN for testing.

```
Future<String> generateDebugPin()
```

Example:

```
final pin = await _orchestration.generateDebugPin();
print('Debug PIN: $pin');
```

⚠ Instructions Callback:

`generateDebugPin` may also trigger instructions through the event stream with type 'instructions' (implementation varies by platform).

6. startMobileApproveJourney

Start mobile approval journey.

```
Future<void> startMobileApproveJourney({
  required Map<String, dynamic> payload,
  required Map<String, dynamic> startJourneyOptions,
})
```

Example:

```
await _orchestration.startMobileApproveJourney(
  payload: {'sessionId': 'session-123'},
  startJourneyOptions: {'encrypted': true},
);
```

⚠ **Instructions Callback:**

Similar to submitClientResponse, startMobileApproveJourney may also trigger instructions through the event stream with type 'instructions'.

7. setLoggingEnabled

Enable or disable SDK logging.

```
Future<bool> setLoggingEnabled(bool enabled)
```

Example:

```
await _orchestration.setLoggingEnabled(true); // Enable for debugging
await _orchestration.setLoggingEnabled(false); // Disable for production
```

8. setPushToken

Set the push notification token for the SDK.

```
Future<void> setPushToken(String token)
```

Parameters:

- token (String): The push notification token from your push notification service

Example:

```
// Get token from Firebase/APNs and set it
final token = await FirebaseMessaging.instance.getToken();
if (token != null) {
  await _orchestration.setPushToken(token);
}
```

9. journeyResponseStream ⚠ MANDATORY

Stream for real-time journey events. **This is the ONLY way the SDK communicates with your app.**

```
Stream<Map<String, dynamic>> get journeyResponseStream
```

⚠ **CRITICAL REQUIREMENT:**

- You MUST listen to this stream BEFORE starting any journey
- Without this listener, your app will not receive any responses from the SDK

- All authentication results, UI instructions, and errors are delivered through this stream

Response Types You'll Receive:

- action:information - Display information to user
- login_form - Show login form
- collect_totp_form - Show TOTP/OTP input
- action:rejection - Authentication failed
- success - Journey completed successfully
- instructions - Native SDK instructions (triggered by submitClientResponse, startMobileApproveJourney, generateDebugPin)

Example:

```
// Set up listener BEFORE starting journeys
orchestration.journeyResponseStream.listen((response) {
  // Handle instructions from submitClientResponse
  if (response['type'] == 'instructions') {
    final instructions = response['instructions'];
    print('Instructions received: $instructions');
    // Process additional guidance or configuration
    return;
  }

  if (response['success'] == true) {
    final stepId = response['response']?['journeyStepId'];
    print('Current step: $stepId');

    // Handle different journey steps
    switch (stepId?.toLowerCase()) {
      case 'action:information':
        _showInformationScreen(response['response']);
        break;
      case 'login_form':
        _showLoginForm(response['response']);
        break;
      // ... handle other steps
    }
  } else {
    print('Journey error: ${response['error']}');
  }
});
```

Error Handling

Common Error Patterns

```

try {
  await _orchestration.startJourney(journeyId: 'test-journey');
} on PlatformException catch (e) {
  switch (e.code) {
    case 'invalidArguments':
      print('Invalid parameters provided');
      break;
    case 'sdkInitError':
      print('SDK not initialized');
      break;
    case 'startJourneyError':
      print('Failed to start journey: ${e.message}');
      break;
    default:
      print('Unknown error: ${e.message}');
  }
} catch (e) {
  print('Unexpected error: $e');
}

```

Response Validation

```

void _validateResponse(Map<String, dynamic> response) {
  if (response['success'] != true) {
    final error = response['error'] ?? 'Unknown error';
    throw Exception('Journey failed: $error');
  }

  final responseData = response['response'];
  if (responseData == null) {
    throw Exception('Invalid response data');
  }
}

```

Best Practices

1. Initialization

- Initialize SDK once when app starts
- Handle initialization errors gracefully
- Use configuration files for production apps

2. Journey Management CRITICAL

- **MANDATORY:** Always listen to journeyResponseStream BEFORE starting any journey
- The SDK communicates exclusively through this stream - without it, your app won't receive any responses
- Handle all possible journey steps in your response handler
- Implement proper loading states during journey execution

What happens without `_listenToResponses()`:

- Your app will start journeys but receive NO notifications when they complete
- Users will be stuck on loading screens with no way to proceed
- Authentication results, errors, and UI steps will be lost
- The journey will appear to "hang" indefinitely

3. Error Handling

- Implement comprehensive error handling for all API calls
- Provide user-friendly error messages
- Log errors for debugging (disable in production)

4. Security

- Never hardcode sensitive credentials in your app
- Use secure storage for configuration data
- Enable logging only during development

5. Performance

- Cancel stream subscriptions when not needed
- Avoid multiple simultaneous journey executions
- Cache SDK initialization state

Troubleshooting

iOS Issues

Problem: Build fails with missing frameworks

Solution: Ensure iOS deployment target is 13.0+

Problem: Camera/Location permissions denied

Solution: Add proper usage descriptions in Info.plist

Android Issues

Problem: Network security errors

Solution: Add network security config for HTTP domains (if needed)

Problem: Proguard issues in release builds

Solution: Add proguard rules for Transmit Security SDK

Common Issues

Problem: "SDK not initialized" error

Solution: Call `initialize()` or `initializeSDK()` before other operations

Problem: Journey responses not received / Journey appears to hang

Solution: You MUST subscribe to `journeyResponseStream` before starting journeys

Symptoms:

- Journey starts but never completes
- Loading screens that never end
- No error messages or success notifications

- App appears frozen during authentication

Fix:

```
// Correct order:
_listenToResponses(); // 1. Set up listener FIRST
await _startJourney(); // 2. Then start journey

// Wrong order:
await _startJourney(); // Journey will hang!
_listenToResponses(); // Too late - responses already lost
```

Problem: Invalid journey step handling

Solution: Implement handlers for all expected journey steps

Platform Differences

Login Form Response ID: The response ID for login form submissions differs between platforms:

```
import 'dart:io' show Platform;

// Use platform-specific response ID
final responseld = (Platform.isAndroid) ? 'password' : 'Password';

await orchestration.submitClientResponse(
  responseld: responseld,
  data: {
    'username': username,
    'password': password,
    'email': email,
  },
);
```

Example App Structure

```
lib/
├── main.dart           # App initialization
├── services/
│   └── orchestration_service.dart # SDK wrapper
├── screens/
│   ├── login_screen.dart  # Login form
│   ├── info_screen.dart   # Information display
│   └── pin_screen.dart     # PIN input
├── utils/
│   └── journey_handler.dart # Journey response logic
```

This guide covers everything needed to integrate the Flutter TS Identity Orchestration plugin. For additional support, refer to the official Transmit Security documentation.